

Web Application Exploitation response

Web / API · **Critical** severity · SOC IR Playbook

Incident type	Application-layer Attack
Severity	Critical
Priority	High
Detection sources	Web Application Firewall (WAF), SIEM, IDS/IPS, Web server logs, Cloud monitoring tools

Scenario

An attacker exploits a vulnerability in a web application or server to gain unauthorised access, execute commands or extract sensitive data. The attack may be detected via WAF alerts, SIEM logs or anomalous behaviour.

MITRE ATT&CK

T1059, T1190, T1505

Preparation

Conduct regular vulnerability assessments Use tools like Nexpose, Tenable or Burp Suite Implement a WAF Configure OWASP top 10 rule sets (e.g., ModSecurity, Cloudflare, AWS WAF) Log HTTP traffic Ensure proper logging from web servers, app servers and proxies Patch management Automate patch cycles for web frameworks, plugins and platforms Code review & DevSecOps integration Integrate SAST/DAST tools into CI/CD pipeline

Detection & Analysis

Alert from WAF or SIEM SQL injection, RCE, XSS, LFI/RFI attempts Review logs Analyse HTTP requests, server responses, unusual error codes (e.g., 500, 403) Validate input payloads Confirm attack vector via payload (e.g., ' OR 1=1--', <?php system(\$_GET[cmd]) ?>) Check for compromise Look for shell uploads, privilege escalations, abnormal process execution

Containment

Block attacker IPs Use WAF, firewall or reverse proxy to block source IP Disable affected web functions Temporarily shut down vulnerable modules or APIs Isolate the application server Disconnect from internal network if lateral movement is suspected Revoke session tokens If user sessions or cookies are believed to be hijacked

Eradication

Remove malicious scripts or shells Search for web shells, reverse shell listeners or backdoors Patch exploited vulnerability Update code, platform, plugin or misconfiguration Harden application Implement input validation, sanitisation, parameterised queries Scan entire application stack Revalidate with updated vulnerability scanner to confirm fix

Recovery

Restore services Bring application back online after confirming clean state Monitor post-restoration Closely observe logs for repeat attempts or backdoor access Notify affected users or customers If data breach occurred, comply with disclosure requirements Conduct retest Confirm no residual access or re-exploitation risk exists

Lessons Learned & Reporting

Perform root cause analysis Identify coding flaw, misconfiguration or patch delay Update SIEM and WAF rules Add custom detections based on observed exploit vectors Improve secure coding practices Conduct refresher training for developers on OWASP Top 10 Document incident timeline Include detection time, TTPs, impact and mitigation steps Report as required If personal data was affected, report to regulators or customers

Tools typically involved

WAF (e.g., ModSecurity, AWS WAF, Cloudflare); SIEM (e.g., Splunk, Sentinel, QRadar); Web server logs (e.g., Apache, Nginx); Vulnerability scanners (e.g., Nessus, Qualys, Nikto); EDR/XDR (if lateral movement occurred); Forensics tools (if shell or system compromise suspected)

Success metrics (SLA targets)

Detection Time <5 minutes from WAF/SIEM alert Containment Time <30 minutes from confirmation Vulnerability Patch Time <24 hours (critical) or <7 days (high) Post-Incident Retest Time Within 48 hours after recovery